

CH04 用户管理与进程管理

考试复习笔记

来源：开源操作系统课件（46 页）+ 用户管理命令 + 进程管理命令

日期：2026 年 6 月 23 日

章节总览

章节	内容
第一部分	用户管理基本概念（多用户系统、UID/GID、关键文件）
第二部分	su/sudo 命令与 sudoers 配置
第三部分	进程管理基本概念（生命周期、状态码、资源）
第四部分	进程进阶（fork/exec/system、孤儿/僵尸进程）
附录 A	用户管理命令速查（L04 补充）
附录 B	进程管理命令速查（L04 补充）

1 用户管理基本概念 [p.4–12]

1.1 用户管理的本质——安全边界 [p.4]

Linux 用户管理实际管理的是系统安全边界：

- 哪个身份可以登录系统
- 哪个身份可以访问某些文件
- 哪个身份可以运行某些服务
- 哪个身份可以临时获得管理员权限
- 哪些操作可以被追踪和审计

最小权限原则：日常操作使用普通用户，需要管理员权限时通过 `sudo` 临时提升。每个用户和服务只拥有完成任务所必需的权限。

1.2 多用户系统的目的 [p.5–7]

1. **安全隔离** [p.5]：每个用户有自己的家目录、配置文件和默认权限。alice 默认不能读取 bob 的私人文件。
2. **权限分级** [p.6]：修改 `/etc/passwd`、重启 `sshd`、删除 `/var/log/`、修改 Web 服务目录等操作需要管理员权限。
3. **责任追踪** [p.6]：`sudo` + `sudoers` 机制可以把操作追踪到具体账号，而非笼统的“root 做了某事”。
4. **协作共享** [p.7]：通过用户组（如 `developers`）实现团队对共享目录的访问控制。

1.3 Linux 用户分类（UID） [p.8]

UID 范围	含义	说明
0	root 超级用户	拥有系统最高权限，不受任何权限限制
1–999	系统用户	由系统或服务自动创建（ <code>www-data</code> , <code>nobody</code> , <code>systemd-network</code> ）
1000–60000	普通用户	手动创建的登录用户
65534	<code>nobody</code>	最低权限用户，常用于 NFS 等匿名访问场景

记原则而非死记数字 [p.8]：UID 0 是 root；普通用户和系统用户有不同用途。

1.4 用户与用户组 [p.9–12]

每个用户至少属于一个**主组**（创建时自动创建同名组），可以加入多个**附加组**。

1.4.1 关键文件 [p.10–12]

文件	内容	权限
/etc/passwd	用户账户信息（用户名、UID、GID、家目录、Shell）	所有用户可读
/etc/shadow	用户密码哈希及密码策略	仅 root 可读
/etc/group	用户组信息（组名、GID、组成员列表）	所有用户可读
/etc/gshadow	组密码及管理员信息	仅 root 可读
/etc/skel/	新用户家目录模板文件	—
/etc/login.defs	创建用户时的默认策略	所有用户可读
/etc/shells	系统认可的登录 Shell 列表	所有用户可读

1.4.2 /etc/passwd 字段解析 [p.11]

```
alice:x:1000:1000:Alice Zhang:/home/alice:/bin/bash
```

字段	示例值
1. 用户名	alice
2. 密码占位符	x（实际密码在 /etc/shadow）
3. UID	1000
4. 主组 GID	1000
5. 用户描述	Alice Zhang
6. 家目录	/home/alice
7. 登录 Shell	/bin/bash

1.4.3 /etc/shadow 字段解析 [p.12]

```
alice:$6$rounds=656000$salt...hash...:19800:0:99999:7:::
```

字段	含义
1	用户名
2	加密后的密码哈希（\$6\$ = SHA-512）
3	上次修改密码日期（距 1970-01-01 天数）
4	密码最短修改间隔（天）
5	密码最长有效期（天），99999= 永不过期
6	密码过期前警告天数
7	密码过期后账户禁用天数
8	账户过期日期
9	保留字段

2 su 与 sudo 命令 [p.15–21]

2.1 su vs sudo 对比 [p.15, 20]

特性	su	sudo
需要的密码	目标用户的密码（通常是 root）	当前用户自己的密码
权限粒度	切换到整个用户会话	可精确到单条命令
审计追踪	切换后无法区分是谁操作的	每条命令都有日志记录
权限回收	需修改密码	删除 sudo 规则即可
安全性	较低（共享密码）	较高（最小权限、可审计）
适用场景	临时完全切换身份	日常特权操作

2.2 su 命令用法 [p.15]

```
su -           # 切换到 root（登录式，加载环境变量）
su - bob      # 切换到 bob 用户（登录式）
su bob        # 不加 -, 不加载目标用户环境
```

2.3 sudo 命令用法 [p.15]

```
sudo apt update           # 以 root 权限执行单条命令
sudo systemctl restart nginx # 以 root 权限管理服务
sudo -u www-data cat /var/log/nginx/access.log # 以指定用户身份执行
sudo -i                   # 以 root 身份打开交互式 Shell
```

2.4 sudoers 配置语法 [p.18]

```
# 用户 主机=(以谁的身份:以哪个组的身份) 命令
alice ALL=(ALL:ALL) ALL # alice 可执行所有命令
%devops ALL=(ALL:ALL) ALL # devops 组可执行所有命令
bob ALL=(ALL) /bin/systemctl restart nginx # bob 只能重启 nginx
deploy ALL=(ALL) NOPASSWD: /bin/systemctl restart myapp # 无需密码
alice ALL=(www-data) ALL # 以 www-data 身份执行
```

关键规则 [p.18]:

- 必须使用 `visudo` 编辑 `/etc/sudoers`（自动检查语法、防并发编辑）
- NOPASSWD 谨慎使用，只对特定自动化脚本开放

2.5 生产环境最佳实践 [p.21]

1. 禁用 root 直接登录: `PermitRootLogin no` (`sshd_config`)
2. 严格限制 sudo 权限: 只允许执行必要的命令, 不给 ALL

3. NOPASSWD 要谨慎：只对特定命令（自动化部署脚本）
4. 定期审计 sudo 使用：查看 /var/log/auth.log 或 /var/log/secure

2.6 sudo 审计 [p.19]

查看 sudo 使用日志

cat /var/log/auth.log | grep sudo # Debian/Ubuntu

grep sudo /var/log/secure # CentOS/RHEL

日志示例：

May 07 10:30:01 server sudo: alice : TTY=pts/0 ;

PWD=/home/alice ; USER=root ; COMMAND=/bin/systemctl restart nginx

3 进程管理基本概念 [p.23–32]

3.1 程序 vs 进程 vs 线程 [p.23]

概念	性质	特点
程序	静态	存放在磁盘上的可执行文件或脚本
进程	动态，有 PID	程序运行起来后的实例
线程	共享进程资源	进程内部的执行流

3.2 进程核心属性 [p.24]

属性	说明
PID	进程唯一标识号，由内核按顺序分配
PPID	父进程的 PID（1 表示父进程是 systemd/init）
UID/GID	运行该进程的用户和组
状态	进程当前状态（R/S/D/T/Z）
优先级	进程调度优先级（nice 值，默认 0）
终端	进程关联的控制终端（pts/0、? 表示无终端）

3.3 进程生命周期 [p.25]

创建 (fork) → 就绪 → 运行 → 等待/睡眠 → 终止 → (僵)

3.4 进程状态码（重要考点）[p.26]

状态码	状态名称	说明
R	Running	正在 CPU 上运行或就绪等待调度
S	Sleeping (Interruptible)	可中断睡眠，等待事件（I/O、信号）
D	Disk Sleep (Uninterruptible)	不可中断睡眠，等待磁盘 I/O， kill -9 也无法终止
T	Stopped	被信号（SIGSTOP、Ctrl+Z）暂停
Z	Zombie	进程已终止但父进程尚未调用 wait() 回收
I	Idle	空闲内核线程（新版 ps 显示）

3.5 进程父子关系 [p.27]

- 每个进程都有父进程（PPID 指向父进程 PID）
- 系统启动后第一个进程是 systemd/init（PID 1），是所有用户进程的祖先
- 进程树可通过 `ps tree` 命令查看

3.6 进程与用户权限 [p.28]

以 Nginx 为例：

```
root      900  nginx: master process  # master 以 root 启动（读配置、绑低端口）
www-data  901  nginx: worker process   # worker 切换到低权限处理请求
www-data  902  nginx: worker process
```

安全原则：服务启动阶段可能需要较高权限，处理普通业务时降低权限。

3.7 进程资源监控 [p.29]

指标	含义
%CPU	CPU 使用率
%MEM	内存使用占比
RSS	实际占用的物理内存
VSZ	虚拟内存大小
TIME	累计 CPU 时间
FD	文件描述符（进程打开的文件、管道、socket 等）

3.8 前台/后台/守护进程 [p.30–31]

类型	特征
前台进程	占用当前终端，接收键盘输入，Ctrl+C 中断，Shell 等待其结束
后台进程	不占用终端输入，Shell 立即返回提示符，用 jobs 查看
守护进程 (Daemon)	无控制终端 (TTY=?), 父进程为 PID 1，以 root/系统用户运行，长期运行

4 进程进阶：fork/exec/system [p.34–45]

4.1 fork 函数 [p.34–36]

- fork 启动新进程的方法：复制当前进程
- 在进程表中创建新表项，新表项的许多属性与当前进程相同
- 新进程有自己的数据空间、环境和文件描述符
- 返回值：
 - 父进程中返回子进程 PID
 - 子进程中返回 0
 - 失败返回 -1

4.2 exec 系列函数 [p.37–38]

- 把当前进程替换为新进程（不是创建新进程）
- 新进程由 path 或 file 参数指定
- 常见变体：
 - `execl`：参数列表方式
 - `execvp`：在 PATH 中搜索
 - `execle`：可指定环境变量
- 失败返回 -1，成功不返回

4.3 system 函数 [p.39–40]

fork-and-exec 的封装：`system(string)` 等价于执行 `sh -c string`。

4.4 fork-and-exec 理解命令执行 [p.41–43]

1. 内部命令（如 `cd`, `echo`）：fork 操作，复制父进程
2. 外部命令（如 `ls`, `cat`）：fork-and-exec 操作
3. Shell 脚本：fork-fork-exec 操作

4.5 孤儿进程与僵尸进程 [p.44]

类型	产生原因	影响
孤儿进程	父进程终止 → 子进程成为孤儿 → 被 PID 1 收养	无负面影响，系统自动处理
僵尸进程	子进程终止 → 父进程未调用 <code>wait()</code> 回收	占用 PID 表项，大量积累可能导致 PID 耗尽

4.6 wait 函数 [p.45]

让父进程等待子进程结束后再结束，实现进程同步。

附录 A：用户管理命令速查

本附录内容来自 L04-用户管理相关命令.md，老师强调必须掌握。

4.7 useradd —创建用户

`useradd [-m] [-s shell] [-c 描述] [-u UID] [-r] [-g 主组] [-G 附加组] 用户名`

选项	说明
<code>-m</code>	自动创建用户家目录（ <code>/home/用户名</code> ）
<code>-s</code>	指定登录 Shell（如 <code>/bin/bash</code> ）
<code>-c</code>	用户描述信息（注释）
<code>-u</code>	指定 UID
<code>-r</code>	创建系统用户（ <code>UID < 1000</code> ）
<code>-g</code>	指定主组
<code>-G</code>	指定附加组（可多个，逗号分隔）

4.8 passwd —设置/修改密码

```
passwd [用户名]           # 设置/修改密码（root可改他人）
passwd -l 用户名          # 锁定用户密码
passwd -u 用户名          # 解锁用户密码
passwd -S 用户名          # 查看密码状态
```

4.9 usermod —修改用户

`usermod [-c 描述] [-s shell] [-d 家目录] [-g 主组] [-G 附加组] [-l 新用户名] [-u UID] 用户名`

4.10 userdel —删除用户

```
userdel 用户名           # 删除用户但不删除家目录
userdel -r 用户名        # 删除用户并删除家目录和邮件
userdel -f 用户名        # 强制删除（即使已登录）
```

4.11 查询命令

命令	作用
<code>whoami</code>	显示当前用户名
<code>id</code>	显示当前用户的 UID、GID、所属组
<code>id nginx</code>	显示指定用户信息
<code>who</code>	查看当前登录系统的所有用户
<code>w</code>	比 <code>who</code> 更详细，显示用户正在做什么
<code>last</code>	列出历史登录记录
<code>lastb</code>	列出登录失败的记录（需 <code>root</code> ）

4.12 组管理命令

命令	作用
<code>groupadd 组名</code>	创建新组
<code>groupmod -n 新名旧名</code>	修改组名
<code>groupmod -g GID 组名</code>	修改组 GID
<code>groupdel 组名</code>	删除组
<code>gpasswd -a 用户组</code>	将用户添加到组
<code>gpasswd -d 用户组</code>	将用户从组中移除
<code>gpasswd -A 用户组</code>	设置组管理员

附录 B：进程管理命令速查

本附录内容来自 L04-进程管理相关命令.md，老师强调必须掌握。

4.13 ps 一查看进程

```
ps aux          # BSD 风格，显示所有用户进程
ps -ef         # UNIX 风格，显示所有进程完整信息
ps auxf        # 显示进程树结构
```

字段说明：

字段	含义
USER	进程所有者
PID	进程 ID
%CPU	CPU 使用率
%MEM	内存使用率
VSZ	虚拟内存大小 (KB)
RSS	物理内存大小 (KB)
STAT	进程状态 (R/S/D/T/Z)
START	启动时间
TIME	累计 CPU 时间
COMMAND	命令名

4.14 top 一实时监控

```
top          # 进入交互式进程监控
```

交互命令：

- P：按 CPU 使用率排序
- M：按内存使用率排序
- k：杀死进程（输入 PID）
- q：退出
- 1：切换显示每个 CPU 核心

load average 解读：系统 1 分钟、5 分钟、15 分钟的平均负载。如果 load average > CPU 核心数，说明系统过载。

CPU 状态行：us(用户态) sy(内核态) id(空闲) wa(等待 I/O)

4.15 htop 一增强版 top

```
htop          # 彩色显示，支持鼠标操作（需安装）
```

4.16 pstree —进程树

```
pstree          # 以树状结构显示进程父子关系
pstree -p       # 同时显示 PID
```

4.17 kill —发送信号

```
kill PID        # 发送 SIGTERM (15), 请求正常终止
kill -9 PID     # 发送 SIGKILL (9), 强制终止
kill -1 PID     # 发送 SIGHUP (1), 重载配置
kill -l         # 列出所有信号
```

常用信号表:

信号号	信号名	status 值	说明
1	SIGHUP	129	挂起/重载配置
9	SIGKILL	137	强制终止（不可捕获）
15	SIGTERM	143	正常终止（默认）

最佳实践：先用 `kill PID (SIGTERM)` 给进程清理资源的机会，无效再用 `kill -9`。

4.18 killall / pkill —按名称终止

```
killall nginx    # 杀死所有 nginx 进程
pkill -f "python app" # 按完整命令行匹配
pgrep nginx      # 查找 nginx 的 PID
```

4.19 前后台作业控制

```
command &       # 在后台运行
Ctrl+Z          # 暂停前台进程
bg              # 将暂停的进程放到后台继续运行
fg              # 将后台进程调回前台
jobs            # 查看当前终端的所有作业
kill %n         # 终止第 n 号作业 (n 是 jobs 显示的编号)
```

4.20 nohup 与 disown —脱离终端

```
nohup command & # 忽略 HUP 信号，终端关闭后继续运行
disown -h %1     # 将作业从 Shell 作业列表中移除
```

最佳实践：对于长期运行的服务，优先使用 `tmux` 或 `screen`，比 `nohup` 更灵活。

开源操作系统 · CH04 用户管理与进程管理 · 46 页课件 + 用户管理命令 + 进程管理命令全覆盖